Отчёт по лабораторной работе №10

Основы работы с модулями ядра операционной системы

Авраменко Давид Андреевич

Содержание

1 Цель работы **5**

2 Выполнение лабораторной работы **6**

3 Вывод **16**

4 Контрольные вопросы **18**

5 Список литературы **22**

Список иллюстраций

2.1 Просмотр устройств системы и используемых модулей ядра . . . 7
2.2 Просмотр загруженных модулей ядра в алфавитном порядке . . . 8
2.3 Загрузка модуля ext4 и просмотр информации о нём 9
2.4 Попытка выгрузки модулей ext4 и xfs 10
2.5 Загрузка модуля bluetooth и просмотр его характеристик 11
2.6 Просмотр параметров модуля bluetooth и его выгрузка 12
2.7 Проверка текущей версии ядра и доступных обновлений 12
2.8 Обновление метаданных репозиториев и просмотр списка обновляемых пакетов 13
2.9 Завершение обновления ядра и повторная проверка пакетов . . . 14
2.10 Проверка версии ядра и сведений о системе после обновления . 15

Список таблиц

1 Цель работы

Получить навыки работы с утилитами управления модулями ядра операционной системы.

2 Выполнение лабораторной работы

На виртуальной машине открываю терминал и командой `su` получаю права суперпользователя. Для просмотра устройств, подключённых к системе, и используемых ими модулей ядра выполняю команду `lspci -k`. В выводе отображаются устройства виртуальной машины, включая системные мосты Intel, контроллеры IDE и SATA, видеоадаптер VMware SVGA II Adapter, сетевой контроллер Intel 82540EM Gigabit Ethernet Controller, звуковой контроллер Intel 82801AA AC'97 и контроллеры USB. Строка `Kernel driver in use` указывает драйвер, используемый устройством в данный момент, а `Kernel modules` содержит список подходящих модулей ядра. Для видеоадаптера используется драйвер `vmwgfx`, для сетевого контроллера — `e1000`, для звукового устройства — `snd_intel8x0`, для SATA-контроллера — `ahci`. Также присутствует устройство VirtualBox Guest Service, использующее модуль `vboxguest`. Полученная информация позволяет определить аппаратные компоненты виртуальной машины и связанные с ними драйверы ядра (см. рис. Рисунок 2.1).



Рисунок 2.1: Просмотр устройств системы и используемых модулей ядра

Для просмотра загруженных модулей ядра выполняю команду `lsmod | sort`. Команда `lsmod` выводит список модулей, находящихся в памяти ядра, а `sort` сортирует строки результата в алфавитном порядке. В выводе отображаются названия модулей, занимаемый ими объём памяти и количество использований. Среди загруженных модулей присутствуют `ahci`, отвечающий за работу SATA-контроллера, `e1000`, обеспечивающий работу сетевого адаптера, `ata_piix`, используемый для поддержки IDE-контроллера, и `snd_ac97_codec`, связанный со звуковой подсистемой. Также отображаются модули `dm_mod`, `dm_mirror` и `dm_multipath`, предназначенные для работы подсистемы Device Mapper. Последний столбец показывает модули или компоненты, использующие соответствующий модуль ядра. Например, модуль `snd_ac97_codec` используется модулем `ac97_bus`, а `dm_mirror` зависит от `dm_mod`. Полученный список позволяет определить активные модули и зависимости между ними (см. рис. Рисунок 2.2).

```
root@daavramenko:/home/daavramenko#
root@daavramenko:/home/daavramenko# lsmod | sort
ac97_bus               12288  1 snd_ac97_codec
ahci                   57344  3
ata_generic            12288  0
ata_piix               45056  1
be2iscsi              204800  0
bnx2i                  94208  0
cdrom                  90112  2 isofs,sr_mod
cnic                   90112  1 bnx2i
crc32c_intel           12288  0
crc32_pclmul           12288  0
crct10dif_pclmul       12288  1
cxgb4                 684032  1 cxgb4i
cxgb4i                 69632  0
dm_log                 24576  2 dm_region_hash,dm_mirror
dm_mirror              28672  0
dm_mod                249856  9 dm_multipath,dm_log,dm_mirror
dm_multipath           53248  0
dm_region_hash         28672  1 dm_mirror
drm_ttm_helper         16384  2 vmwgfx
e1000                 200704  0
fuse                  253952  5
ghash_clmulni_intel    16384  0
i2c_dev                28672  0
i2c_piix4              36864  0
i2c_smbus              20480  1 i2c_piix4
intel_pmc_core        147456  0
intel_pmc_ssram_telemetry    16384  1 intel_pmc_core
intel_rapl_common      57344  1 intel_rapl_msr
intel_rapl_msr         20480  0
intel_uncore_frequency_common    16384  0
intel_vsec             28672  2 intel_pmc_ssram_telemetry,pmt_telemetry
iscsi_boot_sysfs       20480  1 be2iscsi
iscsi_tcp              32768  0
isofs                  69632  1
joydev                 28672  0
libahci                69632  1 ahci
```

Рисунок 2.2: Просмотр загруженных модулей ядра в алфавитном порядке

Для проверки наличия модуля файловой системы ext4 выполняю команду `lsmod | grep ext4`. Команда не выводит результатов, поскольку модуль `ext4` отсутствует в списке загруженных модулей. Затем выполняю `modprobe ext4` для его загрузки в ядро. Повторная проверка командой `lsmod | grep ext4` показывает модуль `ext4` размером `1187840` байт, а также связанные с ним модули `mbcache` и `jbd2`, используемые для кэширования и журналирования файловой системы. Для получения подробной информации выполняю `modinfo ext4`. В выводе параметр `filename` указывает путь `/lib/modules/6.12.0-211.16.1.el10_2.0.1.x86_64/kernel/fs/ext4/ext4.ko.xz`, `license: GPL` обозначает лицензию модуля, а `description: Fourth Extended Filesystem` указывает его назначение. Поле `author` содержит сведения о разработчиках, `alias` перечисляет поддерживаемые обозначения файловых систем ext2, ext3 и ext4, а `depends: jbd2,mbcache` показывает зависимости.

Параметр vermagic содержит сведения о совместимости модуля с используемой версией ядра, intree: Y указывает на принадлежность к исходному дереву ядра, retpoline: Y — на наличие соответствующей защиты, а поля signer, sig_key, sig_hashalgo и signature содержат информацию о цифровой подписи. Строки parm, описывающие настраиваемые параметры загрузки, отсутствуют, следовательно, модуль ext4 не предоставляет дополнительных параметров для передачи через modprobe (см. рис. Рисунок 2.3).

```
root@daavramenko:/home/daavramenko# lsmod | grep ext4
root@daavramenko:/home/daavramenko# modprobe ext4
root@daavramenko:/home/daavramenko# lsmod | grep ext4
ext4                  1187840  0
mbcache                 16384  1 ext4
jbd2                   208896  1 ext4
root@daavramenko:/home/daavramenko# modinfo ext4
filename:       /lib/modules/6.12.0-211.16.1.el10_2.0.1.x86_64/kernel/fs/ext4/ext4.ko.xz
license:        GPL
description:    Fourth Extended Filesystem
author:         Remy Card, Stephen Tweedie, Andrew Morton, Andreas Dilger, Theodore Ts'o and others
alias:          fs-ext4
alias:          ext3
alias:          fs-ext3
alias:          ext2
alias:          fs-ext2
rhelversion:    10.2
srcversion:     FD4B974F3ECE1A45E1B9F3B
depends:        jbd2,mbcache
intree:         Y
name:           ext4
retpoline:      Y
vermagic:       6.12.0-211.16.1.el10_2.0.1.x86_64 SMP preempt mod_unload modversions
sig_id:         PKCS#7
signer:         Rocky Linux kernel signing key
sig_key:        1E:24:82:87:11:EB:1E:95:4C:A8:A5:B4:32:2F:EA:F3:72:6C:B8:90
sig_hashalgo:   sha256
signature:      4E:30:B3:16:81:2A:F4:44:8F:22:F8:9A:0B:D5:7E:6A:2D:0D:18:6B:
                11:D1:7F:01:15:16:41:94:2E:D1:21:6A:3E:16:00:C5:46:AF:CE:A0:
                F7:C0:9C:D7:71:56:70:21:40:D7:47:FC:37:24:21:ED:7A:D3:3B:99:
                46:D7:D3:92:40:03:FA:D0:01:94:64:8C:DF:7E:23:BB:A8:38:8F:98:
                F3:43:0B:06:AB:25:89:0C:C1:E9:5F:55:87:20:FC:4C:01:12:F0:F9:
                97:B2:67:61:E7:A1:20:6A:FF:FD:9D:BA:75:A4:CB:69:7E:24:E9:C8:
                EA:A8:C4:D6:DB:71:C3:71:5F:F3:2F:B6:96:DB:F2:98:C5:66:D8:C4:
                31:E8:3F:05:49:E4:1B:51:48:FB:DD:53:E6:CE:3C:4B:4D:1C:ED:C4:
```

Рисунок 2.3: Загрузка модуля ext4 и просмотр информации о нём

Для выгрузки модуля ext4 выполняю команду modprobe -r ext4. Команда завершается без вывода сообщений об ошибках. Повторное выполнение также не сопровождается сообщениями, что свидетельствует об отсутствии препятствий для удаления модуля из памяти ядра. Затем пытаюсь выгрузить модуль файловой системы XFS командой modprobe -r xfs. Система выводит сообщение modprobe: FATAL: Module xfs is in use., означающее, что модуль используется в данный момент и не может быть выгружен. Повторная попытка приводит к аналогичной ошибке. Это связано с тем, что ядро не допускает выгрузку

модулей, необходимых для работы активных компонентов системы, в частности смонтированных файловых систем (см. рис. Рисунок 2.4).

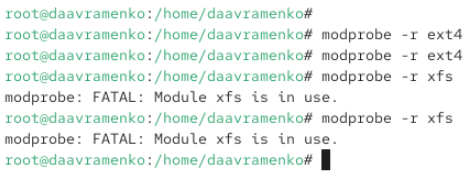


Рисунок 2.4: Попытка выгрузки модулей ext4 и xfs

Для изучения загрузки модулей ядра с параметрами сначала проверяю наличие модуля Bluetooth командой `lsmod | grep bluetooth`. Отсутствие результата означает, что модуль `bluetooth` не загружен. Выполняю команду `modprobe bluetooth`, после чего повторно проверяю список модулей. В выводе появляются `bluetooth` размером `1130496` байт и `rfkill` размером `40960` байт. Модуль `rfkill` обеспечивает программное управление состоянием беспроводных интерфейсов и используется модулем Bluetooth. Затем выполняю `modinfo bluetooth` для просмотра подробных характеристик модуля. Поле `filename` указывает расположение файла `bluetooth.ko.xz` в каталоге `/lib/modules`, `license: GPL` обозначает лицензию, `version: 2.22` — версию Bluetooth Core, а `description: Bluetooth Core ver 2.22` характеризует назначение модуля. В поле `author` указан разработчик Marcel Holtmann, в `depends` приведена зависимость `rfkill`, а в `vermagic` содержится информация о версии ядра, для которой собран модуль. Поля цифровой подписи подтверждают наличие подписи Rocky Linux (см. рис. Рисунок 2.5).

```
root@daavramenko:/home/daavramenko#
root@daavramenko:/home/daavramenko# lsmod | grep bluetooth
root@daavramenko:/home/daavramenko# modprobe bluetooth
root@daavramenko:/home/daavramenko# lsmod | grep bluetooth
bluetooth            1130496  0
rfkill                 40960  4 bluetooth
root@daavramenko:/home/daavramenko# modinfo bluetooth
filename:       /lib/modules/6.12.0-211.16.1.el10_2.0.1.x86_64/kernel/net/bluetooth/bluetooth.ko.xz
alias:          net-pf-31
license:        GPL
version:        2.22
description:    Bluetooth Core ver 2.22
author:         Marcel Holtmann <marcel@holtmann.org>
rhelversion:    10.2
srcversion:     7BD93A2F11B0D86F4E3D358
depends:        rfkill
intree:         Y
name:           bluetooth
retpoline:      Y
vermagic:       6.12.0-211.16.1.el10_2.0.1.x86_64 SMP preempt mod_unload modversions
sig_id:         PKCS#7
signer:         Rocky Linux kernel signing key
sig_key:        1E:24:82:87:11:EB:1E:95:4C:A8:A5:B4:32:2F:EA:F3:72:6C:B8:90
sig_hashalgo:   sha256
signature:      48:9C:D0:40:AA:E7:27:AB:3C:B2:EF:73:C4:43:0F:55:57:7D:DE:3E:
                E3:7C:5E:17:82:15:91:3C:EC:25:5F:1E:A1:74:D1:5B:16:51:0A:FE:
                C3:FC:F8:19:8A:39:89:38:19:AB:35:17:08:2C:78:39:16:A3:A1:6E:
                EB:9E:78:E8:7A:72:C6:14:BD:9E:78:D8:2B:B2:62:6A:49:ED:BB:58:
                8A:C8:A5:37:FC:67:A4:43:07:45:48:F3:58:89:37:2F:EF:29:4E:6B:
                8E:60:9F:4F:60:23:00:C3:FB:2B:F3:18:B0:A8:67:47:C5:CF:F9:C7:
                70:56:88:48:F7:DC:2C:6B:28:8E:DA:60:43:E5:7C:85:2F:AD:3F:17:
                9C:BD:09:36:B1:C3:FB:0D:D0:DB:C4:A4:8C:59:E4:FE:08:02:27:74:
                43:36:67:1F:6D:85:ED:C7:FE:A6:7C:DB:F5:D6:00:08:F9:58:66:00:
                50:E1:EA:8A:96:D0:FE:60:FE:1C:CA:65:09:B4:E7:E2:63:F5:C0:8A:
```

Рисунок 2.5: Загрузка модуля bluetooth и просмотр его характеристик

Продолжаю изучение вывода команды `modinfo bluetooth` и рассматриваю доступные параметры загрузки модуля. Параметр `disable_esco` позволяет запретить создание соединений eSCO, используемых для передачи синхронных аудиоданных по Bluetooth. Параметр `disable_ertm` отключает расширенный режим повторной передачи ERTM, предназначенный для повышения надёжности передачи данных. Параметр `enable_ecred` включает расширенный режим управления потоком на основе кредитов Enhanced Credit Flow Control. Все перечисленные параметры имеют тип `bool` и могут принимать значения `0` или `1`. Например, команда `modprobe bluetooth disable_ertm=1` позволяет загрузить модуль с отключённым режимом ERTM, если модуль предварительно не загружен. В выполненной проверке модуль был загружен без дополнительных параметров. После изучения его характеристик выполняю `modprobe -r bluetooth` для выгрузки модуля. Команда завершается без сообщений об ошибках, что указывает на успешное выполнение операции (см. рис. Рисунок 2.6).

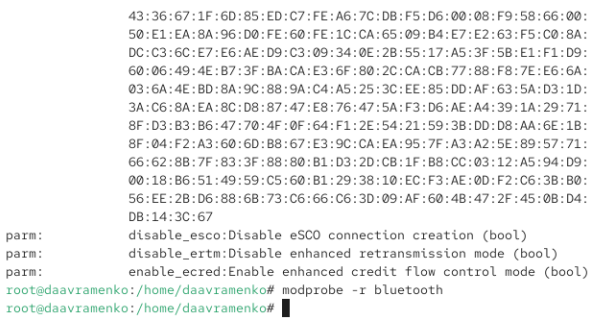


Рисунок 2.6: Просмотр параметров модуля bluetooth и его выгрузка

Для проверки версии используемого ядра выполняю команду `uname -r`. Система выводит значение `6.12.0-211.16.1.el10_2.0.1.x86_64`, которое обозначает текущую версию ядра Linux для архитектуры x86-64. Затем выполняю `dnf list kernel` для просмотра установленных и доступных пакетов ядра. Менеджер пакетов обращается к репозиториям Rocky Linux 10 `BaseOS`, `AppStream` и `Extras`. В разделе `Installed Packages` отображается установленный пакет `kernel.x86_64` версии `6.12.0-211.16.1.el10_2.0.1`, а в разделе `Available Packages` доступна более новая версия `6.12.0-211.68.1.el10_2`. Сравнение версий показывает возможность обновления ядра операционной системы до актуальной версии из подключённых репозиториев (см. рис. Рисунок 2.7).

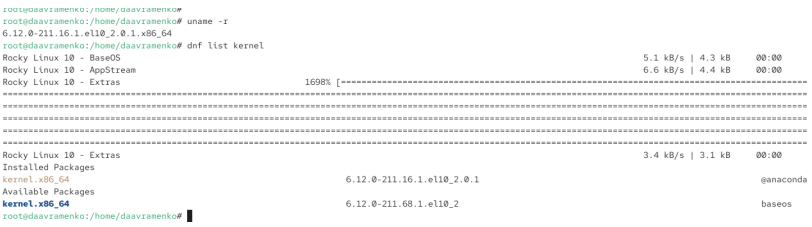
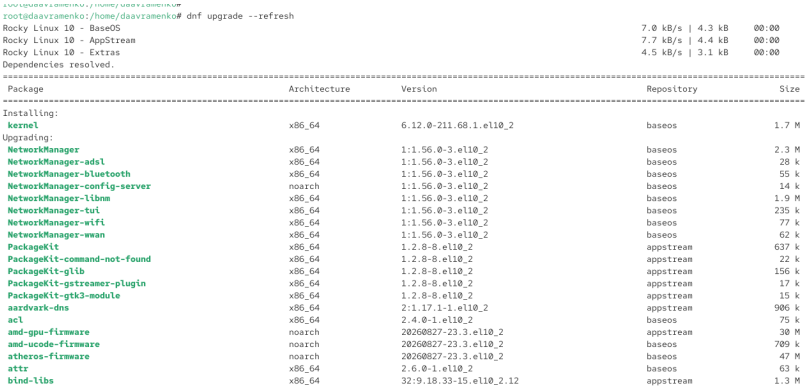


Рисунок 2.7: Проверка текущей версии ядра и доступных обновлений

Перед обновлением ядра выполняю команду `dnf upgrade --refresh`, которая принудительно обновляет метаданные подключённых репозиториев и проверяет наличие обновлений установленных пакетов. После получения информации из репозиториев `BaseOS`, `AppStream` и `Extras` менеджер пакетов разрешает зависимости и формирует список изменений. В разделе `Installing`

отображается новое ядро kernel версии 6.12.0-211.68.1.el10_2 для архитектуры x86_64. В разделе Upgrading перечислены пакеты, для которых доступны обновления, включая компоненты NetworkManager, NetworkManager-bluetooth, NetworkManager-wifi, PackageKit, acl, bind-libs и другие системные библиотеки и утилиты. В таблице указываются архитектура пакета, устанавливаемая версия, репозиторий и размер загрузки. Обновление системы необходимо для устранения известных ошибок, получения исправлений безопасности и обеспечения совместимости компонентов операционной системы с новой версией ядра (см. рис. Рисунок 2.8).



```
root@daavramenko:/home/daavramenko# dnf upgrade --refresh
Rocky Linux 10 - BaseOS                                          7.0 kB/s | 4.3 kB     00:00
Rocky Linux 10 - AppStream                                       7.7 kB/s | 4.4 kB     00:00
Rocky Linux 10 - Extras                                          4.5 kB/s | 3.1 kB     00:00
Dependencies resolved.
================================================================================
 Package                  Architecture  Version                  Repository  Size
================================================================================
Installing:
 kernel                   x86_64        6.12.0-211.68.1.el10_2   baseos     1.7 M
Upgrading:
 NetworkManager           x86_64        1:1.56.0-3.el10_2        baseos     2.3 M
 NetworkManager-adsl      x86_64        1:1.56.0-3.el10_2        baseos      28 k
 NetworkManager-bluetooth x86_64        1:1.56.0-3.el10_2        baseos      55 k
 NetworkManager-config-server noarch    1:1.56.0-3.el10_2        baseos      14 k
 NetworkManager-libnm     x86_64        1:1.56.0-3.el10_2        baseos     1.9 M
 NetworkManager-tui       x86_64        1:1.56.0-3.el10_2        baseos     235 k
 NetworkManager-wifi      x86_64        1:1.56.0-3.el10_2        baseos      77 k
 NetworkManager-wwan      x86_64        1:1.56.0-3.el10_2        baseos      62 k
 PackageKit               x86_64        1.2.8-8.el10_2           appstream  637 k
 PackageKit-command-not-found x86_64    1.2.8-8.el10_2           appstream   22 k
 PackageKit-glib          x86_64        1.2.8-8.el10_2           appstream  156 k
 PackageKit-gstreamer-plugin x86_64     1.2.8-8.el10_2           appstream   17 k
 PackageKit-gtk3-module   x86_64        1.2.8-8.el10_2           appstream   15 k
 aardvark-dns             x86_64        2:1.17.1-1.el10_2        appstream  906 k
 acl                      x86_64        2.4.0-1.el10_2           baseos      75 k
 amd-gpu-firmware         noarch        20260827-23.3.el10_2     appstream   30 M
 amd-ucode-firmware       noarch        20260827-23.3.el10_2     baseos     709 k
 atheros-firmware         noarch        20260827-23.3.el10_2     baseos      47 M
 attr                     x86_64        2.6.0-1.el10_2           baseos      63 k
 bind-libs                x86_64        32:9.18.33-15.el10_2.12  appstream  1.3 M
```

Рисунок 2.8: Обновление метаданных репозиториев и просмотр списка обновляемых пакетов

После выполнения обновления проверяю результат установки пакетов ядра. В выводе команды dnf upgrade --refresh отображается сообщение Complete!, подтверждающее завершение обновления. В разделе Installed перечислены пакеты kernel, kernel-core, kernel-devel, kernel-modules, kernel-modules-core и kernel-modules-extra версии 6.12.0-211.68.1.el10_2.x86_64. Основной пакет kernel обеспечивает установку ядра, kernel-core содержит его основные компоненты, kernel-devel предоставляет файлы для сборки дополнительных модулей, а пакеты kernel-modules, kernel-modules-core и kernel-modules-extra содержат различные группы модулей и драйверов. Затем выполняю команду dnf update kernel для проверки обновлений ядра.

Система выводит `Nothing to do.` и `Complete!`, подтверждая отсутствие дополнительных доступных обновлений. Аналогичные сообщения получаю после выполнения `dnf update` и повторной команды `dnf upgrade --refresh`. Это означает, что все доступные обновления из подключённых репозиториев уже установлены (см. рис. Рисунок 2.9).

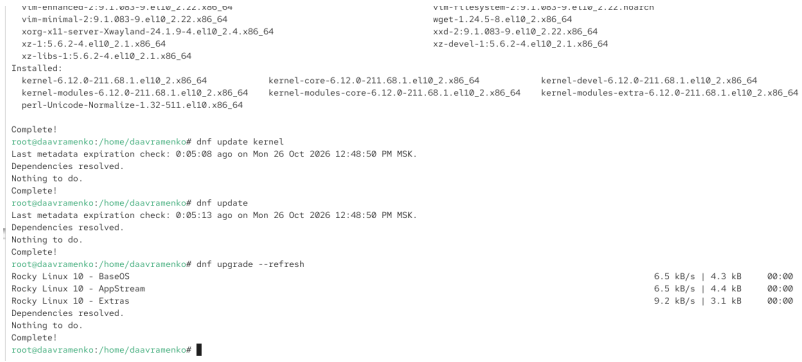


Рисунок 2.9: Завершение обновления ядра и повторная проверка пакетов

После установки обновлений перезагружаю виртуальную машину для запуска операционной системы с новой версией ядра. Открываю терминал и выполняю команду `uname -r`. В выводе получаю `6.12.0-211.68.1.el10_2.x86_64`, что подтверждает успешный запуск обновлённого ядра вместо ранее использовавшейся версии `6.12.0-211.16.1.el10_2.0.1.x86_64`. Затем выполняю команду `hostnamectl` для просмотра сведений об операционной системе и окружении. В выводе `Static hostname: daavramenko` указывает имя компьютера, `Operating System: Rocky Linux 10.2 (Red Quartz)` — название и версию дистрибутива, `Kernel: Linux 6.12.0-211.68.1.el10_2.x86_64` — используемое ядро, а `Architecture: x86-64` — архитектуру системы. Параметры `Virtualization: oracle`, `Hardware Model: VirtualBox` и `Hardware Vendor: innotek GmbH` подтверждают, что система работает в виртуальной среде VirtualBox. Также отображаются идентификаторы машины и текущей загрузки, сведения о прошивке и дате окончания поддержки операционной системы. Полученные результаты подтверждают успешное обновление ядра и запуск

Rocky Linux с установленной версией `6.12.0-211.68.1.el10_2.x86_64` (см. рис. Рисунок 2.10).

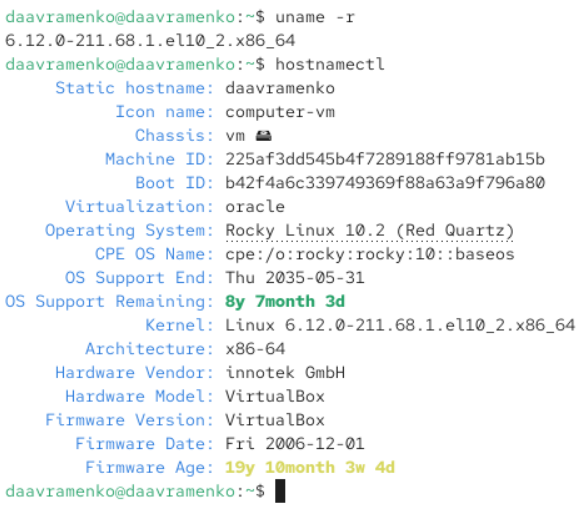


Рисунок 2.10: Проверка версии ядра и сведений о системе после обновления

3 Вывод

В ходе лабораторной работы были изучены основные механизмы управления модулями ядра и обновления ядра операционной системы Rocky Linux. Рассмотрены способы просмотра подключённых устройств, определения используемых ими драйверов, загрузки и выгрузки модулей, а также получения подробной информации об их характеристиках и параметрах.

С помощью команды `lspci -k` получена информация об устройствах виртуальной машины и связанных с ними модулях ядра. Командой `lsmod` просмотрен список загруженных модулей, а с помощью `modprobe` выполнена загрузка модуля файловой системы `ext4`. Команда `modinfo` позволила изучить его характеристики, зависимости и сведения о совместимости с текущей версией ядра.

Также изучена процедура выгрузки модулей. Установлено, что модуль `ext4` может быть выгружен при отсутствии активного использования, тогда как попытка выгрузки `xfs` приводит к ошибке `Module xfs is in use`, поскольку модуль задействован операционной системой.

На примере модуля `bluetooth` рассмотрена возможность загрузки модулей с дополнительными параметрами. Изучено назначение параметров `disable_esco`, `disable_ertm` и `enable_ecred`, позволяющих управлять отдельными функциями Bluetooth.

В заключительной части работы выполнено обновление операционной системы и ядра с помощью менеджера пакетов `dnf`. После установки обновлений и перезагрузки системы командами `uname -r` и `hostnamectl`

подтверждён переход с версии ядра `6.12.0-211.16.1.el10_2.0.1.x86_64` на `6.12.0-211.68.1.el10_2.x86_64`.

В результате работы получены практические навыки управления модулями ядра Linux, определения их зависимостей и параметров, диагностики ошибок при выгрузке модулей, а также установки обновлений ядра операционной системы.

4 Контрольные вопросы

1. **Какая команда показывает текущую версию ядра, которая используется на вашей системе?**

Для просмотра текущей версии ядра операционной системы необходимо выполнить команду:

```
uname -r
```

Параметр `-r` выводит релиз запущенного ядра Linux. Например, после обновления системы команда возвращает значение `6.12.0-211.68.1.el10_2.x86_64`.

2. **Как можно посмотреть более подробную информацию о текущей версии ядра операционной системы?**

Для просмотра подробной информации о ядре используется команда:

```
uname -a
```

Параметр `-a` выводит все доступные сведения, включая название ядра, имя компьютера, релиз и версию ядра, архитектуру процессора и тип операционной системы.

Дополнительно можно использовать команду `hostnamectl`, которая отображает сведения об операционной системе, версии ядра, архитектуре и среде виртуализации.

3. **Какая команда показывает список загруженных модулей ядра?**

Для просмотра списка загруженных модулей ядра необходимо выполнить команду:

```
lsmod
```

Команда выводит таблицу с названиями модулей, занимаемым ими объёмом памяти и количеством использований.

Для сортировки списка по алфавиту используется команда `lsmod | sort`, а для поиска конкретного модуля можно выполнить `lsmod | grep ext4`.

4. **Какая команда позволяет вам определять параметры модуля ядра?**

Для получения информации о параметрах модуля ядра используется команда:

```
modinfo <имя_модуля>
```

Например, для просмотра характеристик модуля Bluetooth необходимо выполнить:

```
modinfo bluetooth
```

Команда отображает путь к файлу модуля, описание, версию, лицензию, зависимости, сведения о цифровой подписи и поддерживаемые параметры.

5. **Как выгрузить модуль ядра?**

Для выгрузки модуля ядра используется команда:

```
modprobe -r <имя_модуля>
```

Например, для выгрузки модуля файловой системы ext4 необходимо выполнить:

```
modprobe -r ext4
```

Параметр `-r` указывает на необходимость удаления модуля из памяти ядра. Выгрузка возможна, если модуль не используется другими компонентами системы.

6. **Что вы можете сделать, если получите сообщение об ошибке при попытке выгрузить модуль ядра?**

Если при выгрузке модуля появляется ошибка `Module is in use`, необходимо определить, какие процессы или другие модули используют его.

Для проверки зависимостей можно выполнить команду:

```
lsmod | grep <имя_модуля>
```

Также можно использовать `modinfo <имя_модуля>` для просмотра информации о модуле.

После этого необходимо остановить использующие модуль службы, освободить связанные с ним ресурсы или выгрузить зависимые модули, если это безопасно.

Например, модуль `xfs` нельзя выгрузить, пока он обслуживает смонтированную файловую систему. Если модуль используется корневой файловой системой, его выгрузка во время обычной работы системы невозможна.

После устранения причины блокировки можно повторить команду `modprobe -r`.

7. **Как определить, какие параметры модуля ядра поддерживаются?**

Для просмотра поддерживаемых параметров модуля используется команда:

```
modinfo -p <имя_модуля>
```

Например:

```
modinfo -p bluetooth
```

Параметр `-p` выводит только список настраиваемых параметров модуля и их описания.

Для модуля `bluetooth` доступны параметры `disable_esco`, `disable_ertm` и `enable_ecred`. Они позволяют управлять отдельными механизмами Bluetooth.

При загрузке модуля значения параметров можно указать непосредственно в команде, например:

```
modprobe bluetooth disable_ertm=1
```

В данном случае модуль загружается с отключённым режимом расширенной повторной передачи ERTM.

8. **Как установить новую версию ядра?**

Для установки новой версии ядра в Rocky Linux используется менеджер пакетов `dnf`.

Сначала необходимо проверить текущую версию ядра:

```
uname -r
```

Затем просмотреть установленные и доступные пакеты ядра:

```
dnf list kernel
```

Для обновления метаданных репозиториев и установленных пакетов выполняется команда:

```
dnf upgrade --refresh
```

После этого можно выполнить обновление ядра:

```
dnf update kernel
```

Для обновления остальных пакетов операционной системы используется команда:

```
dnf update
```

После завершения установки необходимо перезагрузить систему командой `reboot` и при необходимости выбрать новое ядро в меню загрузчика GRUB.

Проверить успешность обновления можно командами:

```
uname -r
```

```
hostnamectl
```

Если в выводе отображается новая версия ядра, значит обновление выполнено успешно и операционная система работает с обновлённым ядром.

5 Список литературы

1. Linux man-pages. modprobe(8) — Add and Remove Modules from the Linux Kernel. — Справочное руководство по загрузке и выгрузке модулей ядра Linux, управлению зависимостями и передаче параметров при загрузке.

2. Linux man-pages. lsmod(8) — Show the Status of Modules in the Linux Kernel. — Документация по просмотру загруженных модулей ядра, анализу их использования и зависимостей.

3. Linux man-pages. modinfo(8) — Show Information About a Linux Kernel Module. — Справочное руководство по получению информации о модулях ядра, их версиях, лицензиях, зависимостях и поддерживаемых параметрах.

4. Red Hat, Inc. Red Hat Enterprise Linux 10. Managing, Monitoring, and Updating the Kernel. — Официальная документация по управлению ядром Linux, установке новых версий, обновлению пакетов ядра и настройке загрузки операционной системы.

5. Rocky Linux Documentation. DNF Package Manager. — Руководство по управлению пакетами в Rocky Linux, работе с репозиториями, проверке доступных обновлений и обновлению операционной системы с помощью менеджера пакетов DNF.